Midterm project presentation

Quantized Neural-Network Acceleration on RV64IMC

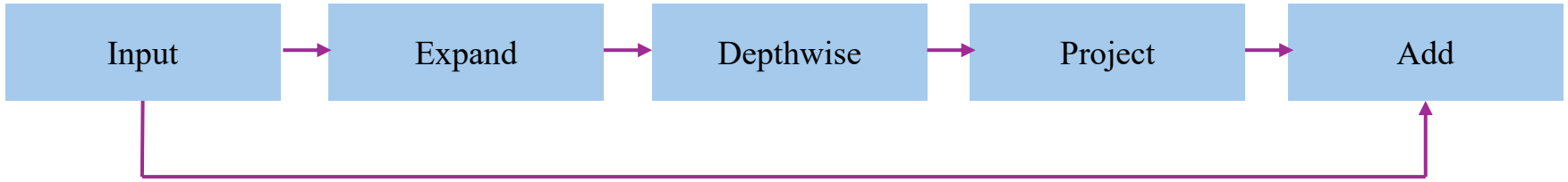
115061629 潘弘恩

Basic guideline

- Evaluate the model and use custom ISA to speedup the model
- Use tiny change to get large performance improve

Efficientnet-Lite0

- Image classification model from Google MediaPipe
- Fully integer with 64 ops
- A MBConv block:



Quantize uint→int8

Stem

1xMBConv1

2xMBConv6

2xMBConv6

3xMBConv6

3xMBConv6

4xMBConv6

1xMBConv6

Head 1x1 conv

Avg pool,Reshape
Fully connected
Softmax
Quantize

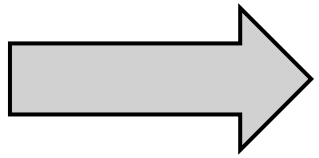
Source:https://huggingface.co/ketiswp/mediapipe-EfficientNet-Lite0-ImageNet-224-int8-uint8-onnx

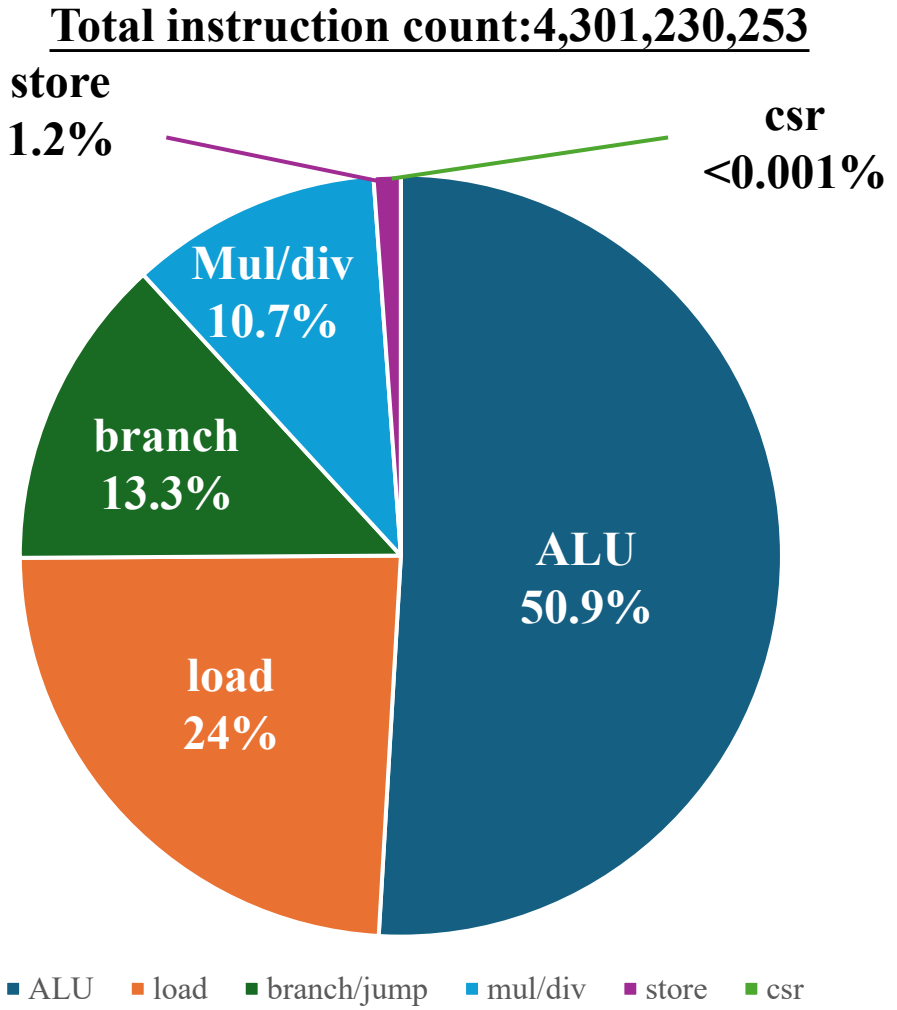
Spike evaluation

Model code (c file) → riscv64-unknown-elf-gcc→spike -g

op	type	instret	cycles	MAC/inst*100	check
0	QUANTIZE	7477124	7477124	0	OK
1	CONV_2D	209824520	209824520	5	OK
2	DWCONV	106118935	106118935	3	OK
3	CONV_2D	77190424	77190424	8	OK
4	CONV_2D	307227466	307227466	6	OK
5	DWCONV	79394204	79394204	3	OK
6	CONV_2D	67454638	67454638	10	OK
7	CONV_2D	143819902	143819902	7	OK
8	DWCONV	118153235	118153235	3	OK
9	CONV_2D	96358806	96358806	11	OK
10	ADD	8561557	8561557	0	OK
11	CONV_2D	143811310	143811310	7	OK
12	DWCONV	58776753	58776753	4	OK
13	CONV_2D	40139123	40139123	11	OK
14	CONV_2D	84019276	84019276	8	OK
15	DWCONV	95262886	95262886	4	OK
16	CONV_2D	64223103	64223103	11	OK
17	ADD	3575617	3575617	0	OK
18	CONV_2D	83854372	83854372	8	OK
19	DWCONV	12246994	12246994	3	OK
20	CONV_2D	32108479	32108479	11	OK
21	CONV_2D	72093640	72093640	10	OK
22	DWCONV	23745909	23745909	3	OK
23	CONV_2D	62214206	62214206	12	OK
24	ADD	1786861	1786861	0	OK
25	CONV_2D	72061269	72061269	10	OK
26	DWCONV	23728111	23728111	3	OK
27	CONV_2D	62214181	62214181	12	OK
28	ADD	1786873	1786873	0	OK
29	CONV_2D	72087038	72087038	10	OK
30	DWCONV	45360924	45360924	5	OK
31	CONV_2D	87098598	87098598	12	OK
32	CONV_2D	134474871	134474871	10	OK
33	DWCONV	63353265	63353265	5	OK
34	CONV_2D	120817045	120817045	12	OK
35	ADD	2499091	2499091	0	OK
36	CONV_2D	134426259	134426259	10	OK

The report using spike -g





Custom ISA

X1

qmac8 rd, rs1, rs2

rs1:activation int8
rs2:weight int8
rd :accumulator int32 →int64

64 bit

rs1: | x7 | x6 | x5 | x4 | x3 | x2 | x1 | x0 |
rs2: | w7| w6| w5| w4 w3| w2| w1| w0 |

$$rd = rd + \sum xi * wi$$

X2

qrequant rd, rs1, rs2

rs1:accumulator result int32
rs2:requant parameters int8
rd :requant result int8

rs2: | act_max | act_min | out_zp | shift | mult (32 bits) |

$$rd = clamp(round(rs1, M) + out_{zp}, act_{min}, act_max)$$

$$where\ M = \frac{mult}{2^{31}} * 2^{Shift}$$

Custom ISA

X1

qmac8 rd, rs1, rs2

rs1:activation int8
rs2:weight int8
rd :accumulator int32 →int64

64 bit

rs1: | x7 | x6 | x5 | x4 | x3 | x2 |
rs2: | w7| w6| w5| w4 w3| w2| w

$$rd = rd + \sum xi * wi$$

Focusing on 1x1 conv

X2

ant rd, rs1, rs2

or result int32
meters int8
int8

min | out_zp | shift | mult (32 bits) |

$d(rs1, M) + out_{zp}, act_{min}, act_max)$

$$\frac{alt}{2^{31}} * 2^{Shift}$$

Why 1x1 conv

1.Dominates the model computation

-87.9% of MACs, 75.5% of B0 instructions

2.It’s a simple general matrix multiply

-fit xqmac8 describtion in project.md

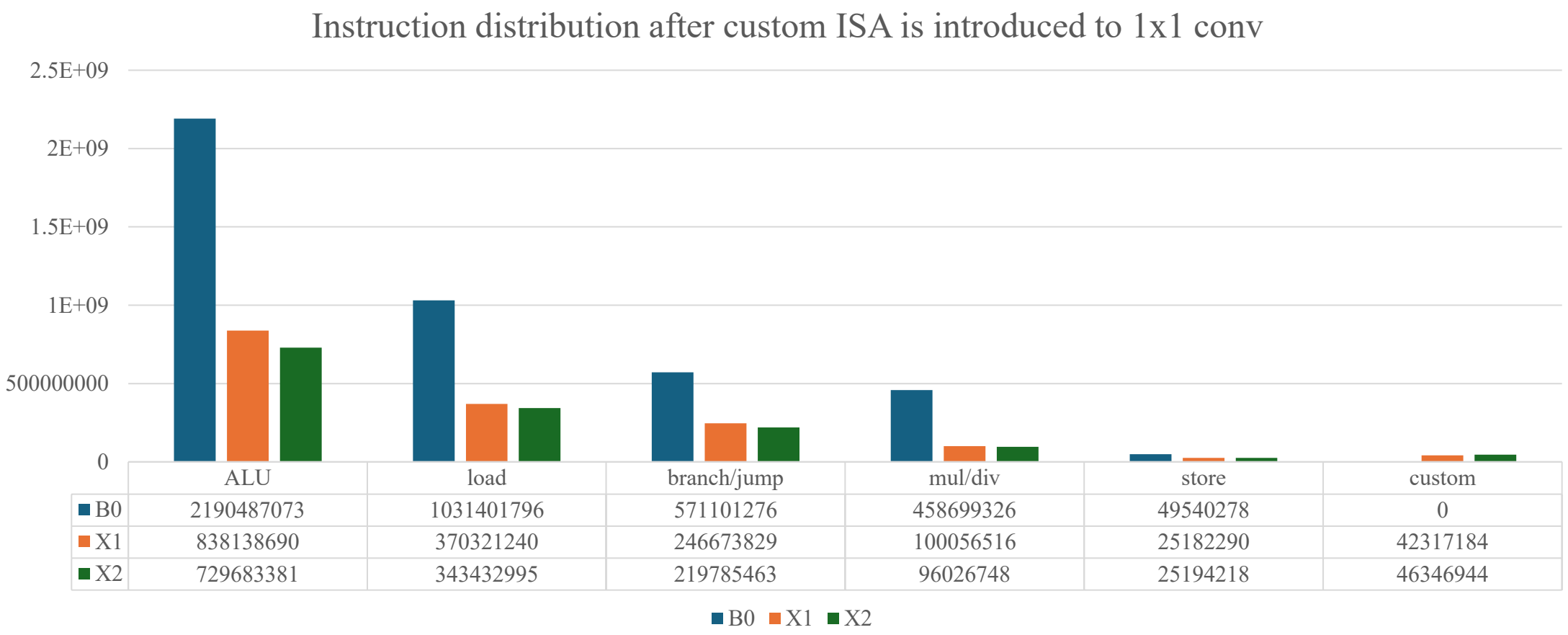
3.Data layout fits naturally

-Cin of all 32 layers is a multiples of 8

-8 activation/weight are contiguous→ no need data rearrangement

-stride = 1, no padding → straightforward

After introducing custom ISA



	ALU	load	branch/jump	mul/div	store	custom
B0	2190487073	1031401796	571101276	458699326	49540278	0
X1	838138690	370321240	246673829	100056516	25182290	42317184
X2	729683381	343432995	219785463	96026748	25194218	46346944

Total instruction count:B0=4,301,230,253 X1=1,622,690,253 X2=1,460,470,253

Performance improvement(by Instruction count)

	B0	X1	X2
1x1 convolution (32 layers)	3218M	**539M**	**377M**
Speedup vs B0	1.00x	5.97x	8.53x
Whole model (w/o softmax)	4264M	1585M	1423M
Whole model speedup vs B0	1.00x	2.69x	3.00x
Depthwise %	18.6%	50.0%	55.7%
Top-1 (golden = 208)	208	208	208

*This table is based on kernel itself, and the data from previous pages include spike from start to end